

WEEK 12 · TYPESCRIPT & REFACTORING

THE COMPILER READS IT TOO

Last week I promised this. Here is the bill.

WHERE WE ARE

Six files, one loop, live data. **The application is finished.**

AFTER TODAY YOU CAN

Move a module to types without changing what it does — and change code's shape on purpose, provably.

WHAT THIS UNLOCKS

The last requirement in the project spec, and the third verb on the exam paper.

**60 minutes of types, 30 of refactoring.
The second is a third of the exam.**

YOU HAVE ALREADY SHIPPED THIS BUG

WEEK 10, AND IT RAN FOR TWO WEEKS

```
1 function addItem(title) {  
2   return {  
3     id: crypto.randomUUID(),  
4     title,  
5     created: new Date(),  
6   };  
7 }
```

1 It works.

2 It survives every click.

3 It breaks on the **first reload**, inside `render`, which is the last place anybody looks for a storage bug.

► `JSON.stringify` calls `Date.prototype.toJSON`. `JSON.parse` has never heard of it.

THE SAME FILE, WITH ONE WORD ADDED

a real localStorage round trip, and the compiler's real answer

↺ reload

open ↗

A DATE IN STATE, AND WHAT A RELOAD DOES TO IT

add an item

save to localStorage

load it back

reset

THE COLLECTION, DRAWN

nothing in the collection yet

THE SAME CODE, TYPED — REAL tsc OUTPUT

```
8
9 function addItem(title: string): Item {
10   return {
11     id: crypto.randomUUID(),
12     title,
13     created: new Date(), // <- the week-10 bug, caught here instead of after a reload
14   };
15 }
```

1 error — line 13

The right-hand panel is loaded from tsc-output.json, captured by running the compiler over tsc-cases/01-date-in-state.ts. Nothing on this page quotes a compiler message from memory.

► Press **ADD** twice — it works. Then **SAVE**, then **LOAD**.

A TYPE IS A CLAIM. IT IS NOT CODE.

the same module, before and after tsc

reload

open

THE SAME MODULE, BEFORE AND AFTER tsc

highlight what disappeared

clear

ts/source.ts — WHAT YOU WRITE

```
// Everything in this file that describes a type is gone after compilation.
export type LoadState = 'idle' | 'loading' | 'error' | 'done';

export interface Item {
  id: string;
  title: string;
  created: string;
}

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null && !Array.isArray(value);
}

export function isItem(value: unknown): value is Item {
  if (!isRecord(value)) return false;
  if (typeof value.id !== 'string' || value.id === '') return false;
  if (typeof value.title !== 'string') return false;
  return typeof value.created === 'string';
}

export function summarise(items: readonly Item[]): { total: number; newest: string | null } {
  const total = items.length;
  const newest = items.reduce<string | null>(
    (best, item) => (best === null || item.created > best ? item.created : best),
    null,
  );
}
```

js/source.js — WHAT THE BROWSER LOADS

```
function isRecord(value) {
  return typeof value === 'object' && value !== null && !Array.isArray(value);
}
export function isItem(value) {
  if (!isRecord(value))
    return false;
  if (typeof value.id !== 'string' || value.id === '')
    return false;
  if (typeof value.title !== 'string')
    return false;
  return typeof value.created === 'string';
}
export function summarise(items) {
  const total = items.length;
  const newest = items.reduce((best, item) => (best === null || item.created > best ?
    item.created : best), null);
  return { total, newest };
}
```

TypeScript never changes the runtime behaviour of JavaScript code.

MOST ANNOTATIONS YOU WRITE ARE ALREADY KNOWN

NOISE

```
1 const name: string = 'קוקו';  
2 const count: number = 3;  
3 const items: string[] = ['a', 'b'];
```

THE SAME THING

```
1 const name = 'קוקו'; // string  
2 const count = 3; // number  
3 const items = ['a', 'b']; // string[]  
4  
5 // here it cannot be inferred:  
6 function shout(word: string): string {  
7     return word + '!';  
8 }
```

Annotate parameters, return types, and anything from outside.
Let inference do the rest.

THE ONES YOU ALREADY HAVE, NAMED

```
1 string number boolean
2 null undefined
3 symbol bigint
4
5 // any      stop checking
6 // unknown  nobody checked yet
7
8 const tags: string[] = []; // any length
9 const pair: [number, number] = [0, 0]; // two
10
11 // worth the ceremony:
12 const [y, m, d] = iso.split('-');
13 // ^ string[], so d may be undefined
```

1 **Seven primitives, and you know all of them**
— this is week 7's list with a compiler attached.

2 **T[]** is “any number of T”. **[A, B]** is
exactly two, and they mean different things.

3 Under **noUncheckedIndexedAccess**, **items[0]** is
T | undefined —
because it is.

SHAPES: interface OR type

interface

A shape other code consumes or extends. The default for an object your application passes around.

type

A union, an intersection, an alias, a tuple — **anything that is not just an object shape.**

THE RULE THAT SETTLES IT

A union **cannot be written as an interface at all.** Where both work, pick one and be consistent.

```
1 interface Item {  
2   id: string;  
3   title: string;  
4   created: string; // ISO 8601. NOT a Date.  
5 }  
6  
7 type ErrorKind = 'offline' | 'timeout' | 'status' | 'shape';  
8  
9 type Ranked = Suggestion & { readonly index: number }; // this, and one more thing
```

OPTIONAL, AND readonly

```
1 interface Item {  
2   id: string;  
3   note?: string;           // may be absent  
4   category: string | null; // there, holding nothing  
5   readonly created: string; // set once  
6 }
```

1 `note?` —
the key may not be there.

2 `| null` — **the key is there and holds nothing.** Use this
for a field you sort or filter by.

3 `readonly` — **free.** One word, and a whole class of bug
becomes a compile error.

► An item with no year is not from year zero, and it has to sort last **IN BOTH DIRECTIONS.**

YOU HAVE BEEN WRITING UNIONS SINCE WEEK 9

TWO BOOLEANS

```
1 let isLoading = false;
2 let isEmpty = false;
3 // four combinations, two of them nonsense
```

ONE FIELD, FOUR VALUES

```
1 type RequestStatus =
2   'idle' | 'loading' | 'done' | 'error';
3 let status: RequestStatus = 'idle';
```

Week 11's argument was that a field with four values cannot be in two of them. Today the compiler enforces which four.

FOUR WAYS TO ASK WHAT A VALUE IS

every check, against every value – computed

↺ reload

open ↗

FOUR WAYS TO ASK WHAT A VALUE IS

run every check against every value

COMPUTED, NOT WRITTEN OUT

AND THE UNION, THROUGH `tsc --strict`

```
1 type ErrorKind = 'offline' | 'timeout' | 'status' | 'shape';
2
3 function sentence(kind: ErrorKind): string {
4   if (kind === 'offline') return 'no route out';
5   if (kind === 'timeout') return 'took too long';
6   if (kind === 'status') return 'they answered, badly';
7   if (kind === 'shape') return 'not what we expected';
8   return kind; // `never` here: every arm is covered, so nothing reaches this line
9 }
10
11 function typo(kind: ErrorKind): string {
12   if (kind === 'timout') return 'took too long'; // <- one letter
13   return '';
14 }
15
```

TS2367: This comparison appears to be unintentional because the types 'ErrorKind' and '"timout"' have no overlap.

line 12 — TS2367. One letter, found before it ran.

► Read the `null` row. **THE FIRST COLUMN SAYS IT IS AN OBJECT.**

THE ONLY HONEST BRIDGE OUT OF `unknown`

```
1 function isItem(value: unknown): value is Item {  
2   if (!isRecord(value)) return false;  
3   if (typeof value.id !== 'string' || value.id === '') return false;  
4   if (typeof value.title !== 'string') return false;  
5   return typeof value.created === 'string';  
6 }  
7  
8 // and what it buys, at the call site:  
9 if (isItem(raw)) {  
10   raw.title.trim();    // `raw` is an Item from here to the end of the block  
11 }
```

`: boolean`

“the answer is yes”

`: value is Item`

“... and from here on you may treat it as one”

A FUNCTION'S TYPE IS ITS CONTRACT

```
1  async function searchTitles(  
2    query: string,  
3    { signal, retries = 1 }: SearchOptions = {},  
4  ): Promise<Suggestion[]> { /* ... */ }  
5  
6  // the return type is not decoration. Delete it and this still compiles:  
7  function toSuggestion(page: unknown) {  
8    if (!ok(page)) return null;  
9    return { title, note, url, index }; // caller now gets `| null` by accident  
10 }
```

WRITE THE RETURN TYPE

A change inside the function that changes what it gives back is caught **in the function**, not three files away.

AN `async` FUNCTION

always returns a `Promise`. So the annotation is `Promise<T>`, never `T`.

A DEFAULT IS NOT OPTIONAL

`retries = 1` means the caller may omit it. Inside, it is a `number`, never `undefined`.

any AND unknown AT THE ONE BOUNDARY THAT MATTERS

the same body, two readers – and what tsc said

↺ reload

open ↗

THE SAME BODY, TWO READERS

body:

THE any READER — NO ANNOTATION

```
const payload = await res.json();  
return payload.query.pages[0].title;
```

THE unknown READER — ONE WORD

```
const payload: unknown = await res.json();  
if (!isPayload(payload)) return 'nothing to show';  
return payload.query.pages[0].title;
```

AND WHAT tsc SAID ABOUT THOSE TWO LINES

```
1 // `res.json()` is typed `Promise<any>` in the DOM lib. That is the whole problem.  
2 async function withAny(res: Response) {  
3   const payload = await res.json();  
4   return payload.query.pages[0].title.toUpperCase(); // no complaint. On a 404 body.  
5 }  
6  
7 async function withUnknown(res: Response) {  
8   const payload: unknown = await res.json();  
9   return payload.query.pages[0].title.toUpperCase(); // <- the compiler asks  
10  | TS18046: 'payload' is of type 'unknown'.  
11 }  
12 // INTENTIONALLY UNCOMPILABLE – this file is a teaching example. Its diagnostics are  
13 // captured by demos/capture-tsc.mjs and shown on the slides; tools/gates.mjs inverts the  
14 // compile gate for it and reports if it ever starts compiling cleanly.
```

► Run the good body first. Then the **404**, which is valid JSON.

as IS A DECLARATION, NOT A CHECK

COMPILES. CHECKS NOTHING.

```
1 const payload =  
2   (await res.json()) as WikiPayload;  
3   payload.query.pages; // may be a 404 document
```

ONE WORD LONGER

```
1 const payload: unknown = await res.json();  
2 if (!isPayload(payload))  
3   throw new ApiError('shape', '...');
```

WHAT YOU WRITE

WHAT IT MEANS

WHAT RUNS

: any

stop checking anything, here and downstream

nothing

: unknown

nobody has checked this yet

nothing — **but you cannot touch it until something does**

as T

I promise it is a T

nothing

value is T

here is the proof

the function body

GENERICS, AND THE SMALLEST USEFUL ONE

```
1 // without: one function per element type, forever
2 function firstString(items: string[]): string | undefined { return items[0]; }
3
4 // with: T is whatever the caller passes, and the return follows it
5 function first<T>(items: readonly T[]): T | undefined { return items[0]; }
6
7 const a = first([1, 2, 3]);           // number | undefined
8 const b = first(['x']);               // string | undefined
9 const c = first<Item>(collection); // Item | undefined
```

A generic is a parameter whose value is a type. That is all it is.

THE GENERIC THAT PROMISES WHAT IT NEVER CHECKS

two signatures, one file, and what `tsc --strict` said about both

↻ reload

open ↗

TWO SIGNATURES THE COMPILER CANNOT TELL APART

the endpoint answers:

`getJSON<T>(url): Promise<T>`

`getChecked<T>(url, guard): Promise<T>`

BOTH OF THEM, THROUGH `tsc --strict`

`tsc --strict`, over both versions: 0 errors.

```
1 interface Suggestion {
2   title: string;
3 }
4
5 // THE SIGNATURE PEOPLE WRITE FIRST. It compiles, and it is a lie: there is no `T` at
6 // runtime, so nothing here checks anything. Every caller gets a fully-typed object that
7 // may be an error document.
8 async function getJson<T>(url: string): Promise<T> {
9   const res = await fetch(url);
10  return res.json();
11 }
12
13 // THE HONEST ONE. The caller supplies the type AND the proof.
14 async function getChecked<T>(url: string, guard: (v: unknown) => v is T): Promise<T> {
15   const res = await fetch(url);
16   const body: unknown = await res.json();
17   if (!guard(body)) throw new Error('the response is not what we asked for');
```

► **VOTE FIRST.** Which of the two is the wrong one?

THE CASTING YOU WILL ACTUALLY NEED

four real diagnostics, and two error messages compared

↺ reload

open ↗

ELEMENT OR NULL, AND NEITHER ONE IS WHAT YOU WANTED

A FORM TO QUERY

search קמח

- first
- second

read the box, both ways

a selector that matches nothing

RAW `querySelector`

`$<T>()` — ONE ASSERTION, IN ONE PLACE

AND WHAT `tsc --strict` SAYS ABOUT THE RAW VERSION

```
1 // `querySelector` returns `Element | null`, and both halves are in the way.
2 function readBox() {
3   const box = document.querySelector('#query');
4   return box.value; // two errors on one line
5 }
6
```

TS18047: 'box' is possibly 'null'.

TS2339: Property 'value' does not exist on type 'Element'.

4 diagnostics over the file — line 4 TS18047, line 4 TS2339, line 21 TS18047, line 21 TS2339. Two of them are on the same line, and they are the two halves of `'Element | null'`.

► Press **A SELECTOR THAT MATCHES NOTHING**. Only one of the two messages names the typo.

AND THE EVENT, WHICH IS THE SAME TRICK

NEITHER OF THESE IS RIGHT

```
1 // does not compile:
2 const row = event.target.closest('li');
3
4 // compiles, and lies:
5 const row = (event.target as HTMLElement)
6   .closest('li');
```

ONE LINE, AND IT IS TRUE AT RUNTIME

```
1 if (!(event.target instanceof Element)) return;
2 // narrowed from here on:
3 const row = event.target.closest('li[data-id]');
```

instanceof is a runtime test the compiler understands. That is **value is T** wearing a different hat.

THE WHOLE BUILD STEP

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "module": "ES2022",
5     "lib": ["ES2022", "DOM", "DOM.Iterable"],
6     "strict": true,
7     "noEmitOnError": true,
8     "rootDir": "ts",
9     "outDir": "js",
10    "declaration": true
11  },
12  "include": ["ts/**/*.ts"]
13 }
```

- 1 `strict` —
without it, none of today happened.
- 2 `noEmitOnError` — a type error must not leave a half-written file for the page to load.
- 3 `declaration` — the `.d.ts` is what gives your **plain JavaScript** files the types without migrating them.

▶ `npx tsc -p tsconfig.json`. That is the build.

HOW YOU MOVE A REAL FILE, WITHOUT STOPPING THE APP

- 1 Types first.**
ts/types.ts — the entity, the unions, the guard.
- 2 One module, and the right one:**
where a value you did not write comes in. That is api.js, every time.
- Rename it and compile.**
3 The compiler emits js/api.js at the same path, so
nothing that imports it changes.
- 4 Fix the first error only**
, and run again. Forty become three.
- 5 Stop.**
The other five stay JavaScript, and get their types from the emitted .d.ts anyway.



LIVE CODING · THE FIRST THREE ERRORS

1 · RENAME

`js/api.js` → `ts/api.ts`, then
`npx tsc -p tsconfig.json`.
Read the count out loud.

2 · FIX THE FIRST ONE

It will be `error.name` in the
`catch`. Add
`error instanceof Error`, run
again, watch the count drop.

3 · THE BOUNDARY

`let payload: unknown;` — one
word. Then let the room tell you
what breaks next.

► **DO NOT FIX EVERYTHING.** Leave it red and say so.

THREE FAULTS IN A MIGRATION THE MACHINE WROTE

```
1  async function getJson<T>(url: string): Promise<T> {  
2    const res = await fetch(url);  
3    return res.json();  
4  }  
5  
6  async function search(q: string) {  
7    try {  
8      return await getJson<Suggestion[]>(URL + q);  
9    } catch (error: any) {  
10     return [];  
11   }  
12 }
```

1

<T> promises what it never checks

2

catch (error: any)

3

no res.ok — **still**

THREE QUESTIONS BEFORE THE BREAK

- 1 You wrote `interface Item` and compiled.
What is in the output file?
- 2 `res.json()` returned a 404 error document.
Which of `any`, `unknown` and `as` told you?
- 3 You migrated one file out of six.
How many are now type-checked?

PART TWO · 30 MINUTES

REFACTORING

A third of the exam, and the only one of the three verbs whose success is invisible.

A refactor is a change to the shape of code that does not change its behaviour.

TWO WORDS, AND BOTH OF THEM DO WORK

BEHAVIOUR-PRESERVING

Every input produces the same output. **Including the inputs you did not think of.**

CHANGE

It is not a rewrite and it is not a tidy-up. Something specific is different, and you can name it.

WHAT IT IS NOT

“I made it nicer.” “I also fixed a small bug while I was in there.”
Both fail the exam question.

- Fixing a bug is good. It is **NOT A REFACTOR**, and on the exam paper the two are different tasks with different wording.

"IDENTICAL", MEASURED INSTEAD OF PROMISED

four versions, 240 inputs, one question

reload

open

FOUR VERSIONS, 240 INPUTS, ONE QUESTION

run every version over every input

show the first difference

DIFFERENCES FROM THE ORIGINAL

V0 — THE ORIGINAL

```
function v0(loan, today) {
  if (loan !== null) {
    if (loan.out) {
      if (loan.due !== null) {
        if (loan.due < today) {
          const [y, m, d] = loan.due.split('-');
          return 'late since ' + d + '/' + m + '/' + y;
        } else {
          const [y, m, d] = loan.due.split('-');
          return 'back on ' + d + '/' + m + '/' + y;
        }
      } else {
        return 'out, no date';
      }
    } else {
      return 'in';
    }
  } else {
    return '-';
  }
}
```

V3 — THE ONE THAT IS NOT A REFACTOR

```
function v3(loan, today) {
  if (!loan?.out) return loan ? 'in' : '';
  if (!loan.due) return 'out, no date';
  const [y, m, d] = loan.due.split('-');
  return `${loan.due < today ? 'late since' : 'back on'} ${d}/${m}/${y}`;
}
```

► Three zeros and one number. **GUESS WHICH VERSION BEFORE YOU LOOK.**

THE CATALOGUE – SEVEN MOVES, AND YOU NEED ALL OF THEM

THE MOVE	THE SMELL THAT CALLS FOR IT
extract function	a comment explaining what the next five lines do
rename for intent	<code>data, temp, handleClick2</code>
remove duplication	the same three lines, twice, with one word different
guard clauses over nesting	the real work is four indents from the margin
split by responsibility	a function whose name needs the word “and”
name the magic value	<code>86400000, '-', 0.6</code>
simplify the conditional	<code>if (x === true) return true; else return false;</code>



EXTRACT, AND RENAME – THE TWO THAT PAY IMMEDIATELY

A COMMENT DOING A FUNCTION'S JOB

```
1 function renderRow(item) {  
2   // turn the ISO date into dd/mm/yyyy  
3   const [y, m, d] = item.due.split('-');  
4   const text = `${d}/${m}/${y}`;  
5   const data = item.number * 0.6; // ???  
6   return `${item.title} - ${text} - ${data}`;  
7 }
```

THE COMMENT BECAME THE NAME

```
1 const DISCOUNT = 0.6;  
2 const formatDate = (iso) =>  
3   iso.split('-').reverse().join('/');  
4  
5 function renderRow(item) {  
6   const cost = item.number * DISCOUNT;  
7   return `${item.title} - ${formatDate(item.due)}`;  
8 }
```

A comment explaining the next lines is a function asking to be born.

SPLIT BY RESPONSIBILITY, AND SIMPLIFY THE CONDITIONAL

ITS NAME NEEDS THE WORD "AND"

```
1 function validateAndSave(item) {  
2   if (item.title === '') return false;  
3   save(item);  
4   if (item.done === true) {  
5     return true;  
6   } else {  
7     return false;  
8   }  
9 }
```

TWO JOBS, TWO FUNCTIONS

```
1 const isValid = (item) => item.title !== '';  
2  
3 function saveItem(item) {  
4   if (!isValid(item)) return false;  
5   save(item);  
6   return item.done;  
7 }
```

► `if (x === true) return true; else return false; is return x;` — and it is in more code than anybody admits.

GUARD CLAUSES, BECAUSE THIS IS THE ONE THEY ALL NEED

THE WORK IS FOUR INDENTS IN

```
1 function label(loan) {  
2   if (loan !== null) {  
3     if (loan.out) {  
4       if (loan.due !== null) {  
5         return format(loan.due);  
6       } else { return 'out, no date'; }  
7     } else { return 'in'; }  
8   } else { return '-'; }  
9 }
```

EVERY EXIT AT THE TOP

```
1 function label(loan) {  
2   if (loan === null) return '-';  
3   if (!loan.out) return 'in';  
4   if (loan.due === null) return 'out, no date';  
5   return format(loan.due);  
6 }
```

The last line is the point of the function, and now you can see it.

HOW TO REFACTOR SAFELY WHEN YOU HAVE NO TESTS

1 One move at a time.

Extract, or rename — not both.

Verify after each.

2 Reload, click the thing, compare the screen.

Ten seconds.

3 Commit after each.

A commit per move is how you find out which move broke it.

4 Write down what must not change

before you start — three behaviours, and how you will check each.

5 If you cannot check it, do not do it.

Cut the scope until you can.

► Step 4 is the one people skip, and it is the one the exam sheet asks for **IN WRITING**.

GOOD, VERSUS MERELY DIFFERENT

#	CRITERION	WHO CAN ANSWER IT
1	behaviour is identical — every input, every output	a machine
2	the thing written twice is now written once	a machine
3	the names say what , not how or where	a person
4	the change is the size of the task, and no larger	a person
5	a future change that touched N places now touches one	a person

These five are on the exam's answer key, in this order. They are not a secret.

FOUR SUBMISSIONS, GRADED IN FRONT OF YOU

the same task, four plausible answers, live scoring

↻ reload

open ↗

THE TASK: REMOVE THE DUPLICATION. BEHAVIOUR IDENTICAL.

THE FIVE CRITERIA, AND WHICH TWO A MACHINE CAN ANSWER

1. Behaviour is identical — every input, every output · a machine can answer this
2. The thing that was written twice is now written once · a machine can answer this
3. The names say what, not how or where · a person must
4. The change is the size of the task and no larger · a person must
5. A future change that used to touch N places now touches one · a person must

grade all four

why did D lose the most?

► RANK THEM BEFORE YOU PRESS GRADE. Then look at D.

LIVE CODING · ONE REFACTOR, THREE COMMITS

1 · NAME THE MAGIC VALUE

`'-'` becomes a constant. Reload.
Commit.

2 · EXTRACT THE DUPLICATION

The date arithmetic becomes
`formatDate`. Reload. Commit.

3 · GUARD CLAUSES

The nesting goes. Reload. Commit.
Then show the three diffs.

► Three commits, three diffs, each one readable in ten seconds. **THAT IS THE WHOLE TECHNIQUE.**

TODAY'S HANDS-ON, AND IT IS NOT A NORMAL ONE

40 MINUTES · THE MIGRATION

`ts/types.ts`, then `ts/api.ts`.
Spec first, tool second, **review third**. This is the project's last requirement.

40 MINUTES · FULL SURGERY

Alone. No AI, no search, no talking. An application you have not seen, three tasks, the exam's wording. Two versions in the room.

10 MINUTES · THE WALKTHROUGH

Answers, **and the grader running on screen** — so you see exactly how the points are awarded.

The three sentences on that paper are word for word the three sentences on the exam.

THE PROJECT IS DUE AT THE END OF THIS WEEK

BEFORE YOU PUSH

`node .checks/run-checks.mjs`
from `project/`. **The same checks**
CI runs, on your machine, in under
a minute.

THE CHECKLIST

`SUBMISSION_CHECKLIST-he.pdf`
— every requirement as something
you **do and observe**, not
something you believe.

THE ONE THAT COSTS THE MOST

Committing `ts/` and forgetting
`js/`. The browser loads the
output. **The grader does too.**

► Moodle: **REPO URL + COMMIT SHA**. The SHA freezes what is graded.

THREE PROMISES, THREE PAYMENTS

PROMISED IN	THE CLAIM	WHAT IT COST
week 9	persistence will be one file, not a rewrite	one line in <code>app.js</code>
week 10	the network will be a layer, not a rewrite	zero lines — and <code>storage.js</code> was never opened
week 11	the compiler will catch a wrong guess about somebody else's response before you run it	you saw it, on slide 3

**Three weeks, three claims, three receipts.
That is what the architecture was for.**

BEFORE NEXT WEEK

- 1 Submit the project.**
Repo URL + SHA, on Moodle, before the deadline. Run the self-check first.
- 2 Open your own project and read it once**
, end to end, as if somebody else wrote it. That is the exam.
- 3 Nothing to install.**
The lab machines will have your project preloaded and no internet. Bring nothing.

Next week you get three sentences you have now read twice. Only the application will be new — and it will be yours.